Oudtra — Technical Materials

Competition: 29th RoboRacer Autonomous Racing Competition, IFAC 2026
Team: Oudtra
Team type: Independent team / personal engineering project
Document version: 1.0
Date: 5 July 2026

---

1. System Overview

Oudtra is developing a 1:10-scale autonomous racing vehicle based on the standard RoboRacer architecture. The vehicle uses a single 2D LiDAR as its primary environment sensor and an NVIDIA Jetson Orin Nano as the onboard computing unit. The autonomous software is organized as a modular ROS 2 pipeline covering sensing, mapping, localization, global raceline generation, local obstacle avoidance, fallback navigation, path tracking, speed planning, safety supervision, and vehicle actuation.

The complete autonomous driving loop is designed to execute onboard the vehicle. Wireless networking is used only for development, visualization, logging, and debugging, and is not required for normal autonomous driving.

---

2. Vehicle Hardware Stack

2.1 Vehicle Platform and Drivetrain

- **Chassis:** Traxxas Fiesta 1:10-scale 4WD platform
- **Traction motor:** Traxxas Velineon 3500 brushless motor
- **Motor controller:** VESC MK VI electronic speed controller
- **Battery:** 3S LiPo battery
- **Battery operation plan:** two separate battery packs are planned for alternating use between runs; only the battery installed on the vehicle is used for propulsion and onboard power during operation
- **Power distribution:** custom self-soldered power distribution board for onboard electronics and subsystem power routing

2.2 Sensing

- **Primary environment sensor:** Hokuyo UST-10LX 2D LiDAR
- **Vehicle state feedback:** wheel-speed/odometry and VESC telemetry through the vehicle interface stack

The LiDAR is the primary source for mapping, particle-filter localization, obstacle detection, and localization-independent reactive fallback navigation.

2.3 Onboard Computing

- **Computing unit:** NVIDIA Jetson Orin Nano, 8 GB

- **Role:** executes localization, perception, planning, control, safety monitoring, and actuation interfaces onboard
- **Deployment:** containerized Linux/ROS 2 software environment for reproducible development and deployment

2.4 Human Interface and Networking

- **Manual development interface:** Logitech F710 gamepad for teleoperation, setup, and permitted start/stop or emergency procedures
- **Wireless networking:** external Wi-Fi router plus backup Wi-Fi dongle for development, visualization, logging, and debugging connectivity between the onboard computer and the development laptop
- **Race-time autonomy:** the autonomous driving pipeline does not depend on Wi-Fi connectivity or remote computation

2.5 Competition Safety Hardware

The vehicle will use the competition-required LiDAR detection box and soft foam bumper when required for multi-vehicle operation. Hardware mounting, wiring, and power distribution are arranged to support safe inspection, maintenance, and rapid battery replacement between permitted sessions.

---

3. Software Stack

3.1 Software Foundation and Architecture

The software is developed in the team's `f1tenth_dev` workspace and builds upon the open-source RoboRacer/F1TENTH software ecosystem, including system-level vehicle interfaces derived from `f1tenth_system` and simulation workflows based on `f1tenth_gym` and `f1tenth_gym_ros`.

The onboard system uses ROS 2 with modular nodes and standard message interfaces. The main functional layers are:

1. Sensor and actuator interfaces
2. Mapping and localization
3. Offline centerline and raceline generation
4. Online obstacle detection and local avoidance planning
5. Localization-independent reactive fallback navigation
6. Global path tracking and speed planning
7. Safety supervision and localization-health handling
8. VESC and steering actuation

Simulation and hardware deployment use closely aligned ROS 2 interfaces so that localization, path following, speed planning, obstacle handling, and safety logic can be tested before onboard deployment.

---

4. Mapping and Localization

4.1 Mapping

The team uses 2D LiDAR-based mapping to generate an occupancy-grid representation of the race circuit. The mapping workflow is intended to support the map formats provided or permitted by the competition and to provide the geometric reference for localization and offline raceline preparation. The software implementation for mapping is based on the ROS2 SLAM library included in the official RoboRacer github project.

4.2 Localization

The primary localization approach is a LiDAR-based particle filter using the occupancy map and real-time laser scans. The localization module publishes the estimated vehicle pose for downstream planning and control. The software implementation for the core particle filter function is based on the ROS2 Particle Filter library included in the official RoboRacer github project.

A separate localization-health assessment layer evaluates whether the pose estimate is sufficiently trustworthy for global-path-based autonomous driving. The raw pose estimate remains available, while the health state is provided to downstream modules so they can select the appropriate driving behavior.

This separation allows the system to recover naturally if localization quality improves again, while preventing downstream planning and control from blindly relying on a temporarily unreliable global pose estimate.

---

5. Global Centerline and Raceline Generation

The team uses an offline path-generation workflow to convert a mapped circuit into a smooth global reference path. The workflow includes:

- extraction of the drivable region from the occupancy map;
- generation and validation of a closed-loop centerline candidate;
- conversion to map/world coordinates;
- resampling and smoothing;
- curvature-based corner analysis;
- optional manual corner key-point adjustment;
- generation of a racing-line offset from the centerline;
- safety-margin constraints relative to the drivable boundary;
- curvature-aware and offset-gradient limiting to reduce path entanglement or abrupt lateral transitions;
- final path smoothing and export for runtime use.

The resulting raceline is stored as an ordered waypoint sequence containing geometric information such as position, heading, and curvature. Runtime ROS 2 nodes publish the reference path and waypoints to the planning and control modules.

---

6. Detection and Avoidance

6.1 Localization Available: Raceline-Referenced Avoidance

When localization is healthy, the vehicle follows the precomputed global raceline. Real-time LiDAR returns are transformed into the map/path reference and checked against a safety corridor around the upcoming segment of the global raceline.

Any scan cluster that intrudes into this corridor is treated as an obstacle, independent of whether it is a static object or another moving vehicle. The local planner then generates a short collision-free bypass trajectory through the available drivable space, tracks that temporary path around the obstacle, and merges the vehicle back onto the global raceline after sufficient clearance is available.

The purpose of this design is to keep the nominal racing behavior simple and fast while activating local replanning only where an actual conflict with the planned path exists.

6.2 Localization Unavailable: LiDAR-Only Reactive Fallback

If global localization is unavailable or judged unreliable, the vehicle switches away from map-referenced raceline tracking. Using the current LiDAR scan directly, the fallback planner identifies the largest safe and reachable forward drivable corridor and commands the vehicle toward that corridor with conservative speed and steering limits.

This fallback mode is designed to keep the vehicle moving safely within locally observed free space without depending on a valid global pose. When localization health recovers sufficiently, the system can transition back to global-raceline-based driving.

---

7. Path Tracking and Speed Control

7.1 Lateral Control

The primary path-following controller is a Pure Pursuit-based controller with speed-dependent lookahead behavior. It tracks either:

- the nominal global raceline; or
- a short local avoidance path when obstacle bypassing is active.

The controller converts the selected path target into steering commands for the vehicle interface.

7.2 Longitudinal Control

The baseline speed planner uses path curvature and configurable safety limits to define a target speed profile. Higher speeds are allowed on low-curvature sections, while speed is reduced for corners and higher-risk path geometry.

The team has also developed an optional PPO-based reinforcement-learning speed module. The learning-based module is not used as an unrestricted end-to-end driver; instead, it produces a bounded speed residual around the rule-based speed target. This preserves the deterministic path-following structure while allowing limited optimization of longitudinal behavior. The rule-based planner remains available as the baseline and fallback speed mode.

---

8. Safety and Degraded-Mode Strategy

The software architecture includes explicit monitoring and fallback behavior rather than assuming all upstream inputs are always valid.

Key safety concepts include:

- localization-health status distributed to planning and control modules;
- fallback from global-raceline tracking to LiDAR-only local corridor driving when localization is unreliable;
- conservative speed limits in degraded operation;
- sensor/data freshness checks in downstream control logic;
- bounded learning-based speed corrections around a deterministic baseline;
- onboard autonomous execution without dependence on remote computation;
- toggle-style emergency braking capability for inspection and permitted emergency use;
- manual gamepad operation restricted to development, setup, and situations permitted by competition procedures.

The overall design principle is to preserve controllable behavior under degraded localization and to provide a clear transition between nominal racing, local obstacle avoidance, and localization-independent fallback driving.

---

9. Development, Simulation, and Validation

The team uses a simulation-first workflow based on the RoboRacer/F1TENTH Gym ROS 2 environment, followed by onboard validation on the physical vehicle.

The workflow includes:

- occupancy-map and path-generation validation;
- visualization and debugging in RViz;
- particle-filter localization evaluation;
- localization-health and recovery testing;
- rule-based raceline tracking validation;
- obstacle-handling and fallback-mode development;
- reinforcement-learning speed training and evaluation in simulation;
- transfer of validated modules to the onboard ROS 2 environment;
- iterative tuning using recorded logs and repeatable test scenarios.

The software is maintained as a modular development workspace so that localization, planning, control, and learning-based modules can be evaluated independently and integrated through ROS 2 topics and parameters.

---

10. Summary

Oudtra's vehicle is a standard RoboRacer/F1TENTH-derived 1:10-scale autonomous racing platform using a Traxxas chassis, Velineon brushless drivetrain, VESC motor control, Hokuyo UST-10LX LiDAR, 3S LiPo power, and an NVIDIA Jetson Orin Nano 8 GB onboard computer.

The software architecture combines LiDAR mapping, particle-filter localization, localization-health monitoring, offline centerline/raceline generation, Pure Pursuit path tracking, curvature-based speed planning, optional bounded reinforcement-learning speed optimization, raceline-referenced obstacle avoidance, and a LiDAR-only largest-drivable-corridor fallback when global localization is not trustworthy.

All autonomous driving computation is intended to run onboard. Wireless communication is reserved for development, visualization, logging, and debugging rather than race-time control.